LfHL: Learning from a Handful of Labels

[1]Arno Demarteau, [1]Elias Hühne, [1]Quentin Geoffroy
[1]University of California, Berkeley

Abstract

Predicting a system's performance on an unseen benchmark from only a handful of its answers is difficult, since information learned elsewhere transfers poorly. We examine which sources of information are useful for this task, using benchmark-held-out cross-validation on 1,720 system–benchmark pairs and the Brier area under the learning curve (ALC) of the Predictive AI Evaluation Competition. The text, metadata, tree-based and language-model features we tried improve ALC by about 0.001 or less. In contrast, a calibrated Bayesian update using a system's own labels reduces ALC from 0.25 to 0.2109, with other prior shapes providing almost no additional gain. Labels from other systems evaluated on the same benchmark provide another source of information. We introduce LfHL, a hierarchical logistic model that pools a benchmark level, tag shifts and per-question shifts across systems. With 2, 5 and up to 20 systems, it achieves ALC scores of 0.2067, 0.2026 and 0.1968, respectively, with improvements in all five folds. Our ablation shows that learning the overall benchmark level accounts for most of the gain at low budgets, while question shifts become the main source of improvement at high budgets when many systems share a benchmark. These gains depend on the number of systems sharing a benchmark. Since development-platform runs score only eight or nine pairs, the gains with few systems are most relevant there. Identical uploads on the platform have a standard deviation of 0.0185, making single scores unreliable for ranking models. Our selected submission uses LfHL fitted on public data only.

1 Introduction

Evaluating every AI system on every benchmark is expensive, while many evaluation decisions only require a reliable estimate of system performance. Item response theory [5, 6, 11] and its recent applications to efficient evaluation [9, 12] have shown that system ability and item difficulty can be estimated from a fraction of the responses *within* a benchmark. The Predictive AI Evaluation Competition (PAIEC) [14] extends this problem to unseen benchmarks for which no training data is available. Specifically, the task is to predict $P(\text{correct})$ for each (system, question) using the question's text, system metadata, and $n \in \{0, 1, 3, 7, 15, 31\}$ revealed answers.

However, benchmarks differ substantially in their formats, domains and definitions of a correct label. As a result, information learned from one benchmark does not necessarily transfer to another. This motivates the central question of our work: *on a new benchmark, where does usable information come from?* We examine three available sources of information and find that their usefulness differs substantially (Figure 1).

Our contributions are:

- **A platform-faithful local evaluation** (Section 2). We implement benchmark-level five-fold cross-validation that reproduces the platform's split, nested budgets, shared `labeled` lists and streaming acquisition. All learned parameters are fitted on training folds only.

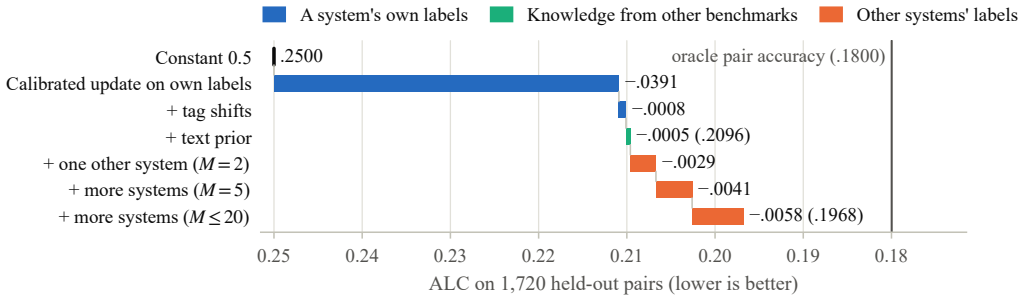


Figure 1: Where usable information comes from on an unseen benchmark. Each step adds to the one above and shows its change in ALC, with the ALC reached in brackets where a source ends. Labels arrive from budget 1, and other systems' labels only if M, the number of systems of one benchmark whose labels are shared, exceeds one. Besides the text prior, the other features we tried from other benchmarks gain about 0.001 or less (Section 4.1).

- **An evaluation of the available information sources, including negative results** (Section 4). We examine what information transfers across benchmarks, how quickly the benefit from a system's own labels saturates, and what additional information other systems' labels provide.
- **LfHL** (Section 3). We introduce a hierarchical logistic model that pools labels across systems and reduces to the own-label model when only one system is available. We further conduct an ablation of the shared terms to identify where the improvement comes from.
- **An analysis of the development platform conditions** (Section 5). We examine how much of the pooling gain can be achieved on the platform and how little can be concluded from a single score.

2 Task, data and evaluation protocol

Task. A *pair* consists of one system s evaluated on one benchmark b containing at least 80 distinct questions. The platform randomly divides each pair's questions into two equally sized pools, one for acquisition and one for evaluation. Acquisition candidates are presented sequentially, with an optional policy (`labeling.py`) determining which labels to reveal. Without such a policy, candidates are sampled uniformly. At each budget $n \in \{0, 1, 3, 7, 15, 31\}$, the model predicts all evaluation questions using the labels revealed so far. The budgets are nested. Let BS_n denote the Brier score [2] at budget n. The score for each pair is

$$\mathrm{ALC} = 0.1\,\mathrm{BS}_0 + 0.2\,(\mathrm{BS}_1 + \mathrm{BS}_3 + \mathrm{BS}_7 + \mathrm{BS}_{15}) + 0.1\,\mathrm{BS}_{31}, \tag{1}$$

which corresponds to the trapezoid rule on $\log_2(n+1)$. The overall suite score is the equally weighted mean across pairs, with constant predictions of 0.5 yielding 0.25. Repeated trials of the same question are treated as separate targets.

The evaluation interface pools labels. The organizers' ingestion code[1] provides *one* `labeled` list at each budget checkpoint, containing all labels purchased so far in the run. Every target at that checkpoint is predicted using the same list. As a result, each target has access to the revealed answers of all other systems in the run, including systems evaluated on the same benchmark. These labels are available at the same budget as the target's own labels, but not at budget 0. However, both official baselines filter `labeled` to retain only labels from the target system, leaving the remaining information unused.

Data. We use only the public Measurement DB [1] for training, fixed at revision `54074453`. Of the 281 benchmark directories, 163 contain binary responses at the question level. We exclude three benchmarks marked `release = withheld`, leaving 160 benchmarks, of which 142 contain a total of 12,985 scoreable pairs. Three observations inform our design:

[1] `tools/streaming_ingestion.py` of the official baseline repository (commit `cc88a95`), lines 147–181.

- *Systems rarely repeat*: 98% of system ids appear in only one benchmark. The available system metadata is also sparse, with `normalized_name` filled for 49% of systems, release date for 33%, and reasoning effort for 0.1%. Furthermore, accuracy on another benchmark matched by system name correlates only 0.23 with accuracy on the target benchmark.
- *Pairs differ a lot*: Pair accuracy has a mean of 0.54 and a standard deviation of 0.27. This corresponds to a spread of 1.62 on the logit scale, or 1.29 relative to each benchmark's mean.
- *Questions do not recur across benchmarks*: Only 0.1% of the 3.9 M distinct public question texts appear in another benchmark, and none appear outside their own benchmark family.

We further calculate in-sample ceilings across all scoreable pairs to illustrate the differences between available information sources. Predicting each benchmark's mean achieves a Brier score of 0.217, compared with 0.174 when using each pair's accuracy and 0.133 when using each question's mean accuracy across systems.

Local evaluation. We implement five-fold cross-validation using `scripts/run_eval.py`, assigning the 160 benchmarks to folds with seed 0. For each fold, we fit all predictors from scratch on the other four folds and evaluate them on the held-out benchmarks. As a result, every benchmark is evaluated exactly once by a model that has not seen it during training. We select the first 20 pairs by system id from each benchmark and evaluate at most 200 questions per pair, resulting in 1,720 pairs across 142 benchmarks. Each pair is randomly divided 50/50 using a seed derived from (benchmark, system). Labels are then revealed in a seeded prefix of the acquisition order, following the platform procedure. To reproduce label pooling, we shuffle the pairs within each benchmark and divide them into *pools* of M systems. Every system in a pool is evaluated using the same shared list of revealed labels at budget n. We also simulate the streaming coordinator to evaluate acquisition policies. Since all methods are evaluated on identical pairs, we compare their performance using paired differences and report 95% confidence intervals obtained through a bootstrap over benchmarks, accounting for correlations between pairs from the same benchmark. We additionally report the number of folds in which each candidate performs better. Our 286 unit tests include equivalence checks that verify, for example, that the fast vectorised scorers used during fitting agree with runtime predictions to 10^{-10}.

3 The model: LfHL

For system s answering question q of benchmark b, LfHL is

$$\operatorname{logit} P(y_{sq} = 1) = a_{sq} + \beta_b + u_s + \sum_{g \in G(q)} \gamma_g + v_q, \tag{2}$$

$$\beta_b \sim \mathcal{N}(0, \rho\,\tau^2), \qquad u_s \sim \mathcal{N}\big(0, (1-\rho)\,\tau^2\big), \qquad \gamma_g \sim \mathcal{N}(0, \sigma^2), \qquad v_q \sim \mathcal{N}(0, \omega^2).$$

All terms beyond the starting logit a_{sq} are learned from `labeled` at prediction time. The benchmark level β_b, tag shifts γ_g and question shifts v_q are shared across systems with revealed labels on the same benchmark, while u_s is specific to each system. The parameter ρ determines how the pair-shift variance is divided between a shared benchmark level and a system-specific component. Question shifts provide information only when at least one system has revealed the corresponding question.

Own-label models are special cases. When only one system is available, $\beta_b + u_s \sim \mathcal{N}(0, \tau^2)$ becomes a single pair shift δ_{sb}. Removing the remaining terms gives the *Bayesian offset*, $\operatorname{logit} P = \ell + \delta_{sb}$. Adding γ_g produces *tag shifts*, while replacing ℓ with a learned a_{sq} gives the *text prior*. With one system, LfHL reproduces this model (tested to 10^{-9}), except that revealed answers also affect evaluation questions with identical visible text. We additionally use the *Beta-binomial* as a conjugate baseline, $p = (\alpha p_0 + k)/(\alpha + n)$ for k correct answers out of n. This corresponds to the posterior mean under a Beta prior [3].

Tag shifts. Many benchmarks provide question-level tags in the form `key=value`, such as `task=qa3` and `lang=ja`. However, their meaning depends on the benchmark, making it difficult to transfer tag weights learned from training benchmarks. Instead, we estimate tag shifts from the revealed questions of the target benchmark. We select grouping keys using only the revealed questions (Appendix B). Tag values without revealed examples retain their prior variance σ^2.

Table 1: Five-fold cross-validated Brier score by budget and ALC on 1,720 pairs of 142 public benchmarks (lower is better), grouped by information source. Δ: paired difference in ALC to the row above (for $M = 2, 5, \leq 20$: to $M = 1$), with a 95% bootstrap interval over benchmarks; folds: number of the five folds in which the row is better. The oracle row knows each pair's true accuracy on its evaluation questions.

Model	BS_0	BS_1	BS_3	BS_7	BS_{15}	BS_{31}	ALC	Δ ALC [95% CI]	folds
Constant 0.5	.2500	.2500	.2500	.2500	.2500	.2500	.2500		
Empirical mean (organizers)	.2500	.3574	.2412	.2066	.1941	.1875	.2436	$-.0064$ [$-.0200$, $+.0059$]	4
Beta-binomial	.2507	.2301	.2134	.1998	.1923	.1871	.2109	$-.0327$ [$-.0372$, $-.0278$]	5
Bayesian offset	.2505	.2299	.2133	.1999	.1924	.1871	.2109	$-.0000$ [$-.0002$, $+.0001$]	3
+ tag shifts	.2505	.2299	.2133	.1991	.1908	.1842	.2101	$-.0008$ [$-.0012$, $-.0004$]	5
+ text prior	.2494	.2290	.2127	.1988	.1907	.1842	.2096	$-.0005$ [$-.0017$, $+.0007$]	4
LfHL, $M = 1$	.2494	.2290	.2127	.1988	.1907	.1842	.2096	$\lvert\Delta\rvert < .0001$ (text twins)	–
LfHL, $M = 2$	.2494	.2241	.2082	.1962	.1890	.1827	.2067	$-.0029$ [$-.0038$, $-.0020$]	5
LfHL, $M = 5$	.2494	.2151	.2036	.1930	.1865	.1799	.2026	$-.0070$ [$-.0088$, $-.0054$]	5
LfHL, $M \leq 20$	.2494	.2055	.1971	.1886	.1815	.1734	**.1968**	$-.0128$ [$-.0150$, $-.0106$]	5
Oracle pair accuracy	.1800	.1800	.1800	.1800	.1800	.1800	.1800		

Starting logit. We train a logistic regression [8] using hashed features [16] from the information available to `predict`. These include 22 hand-made features and hashed tokens extracted from question text, tags and system fields. We weight pairs equally and select regularisation using held-out *benchmarks*. The resulting logit f_{sq} is incorporated as $a_{sq} = \ell + c + \lambda\,(f_{sq} - \mu)$, where the trust parameter λ is learned from out-of-fold logits. Setting $\lambda = 0$ recovers a constant starting logit, allowing the text prior to contribute only when it improves performance on held-out data.

Inference. For each benchmark in `labeled`, we estimate a joint Laplace posterior [15] over (β_b, u, γ) using Newton's method. We then estimate the question shifts v_q separately in one dimension, keeping the other coefficients fixed. For each target, we sum the posterior means of its coefficients and calculate the variance from their posterior covariance, adding the prior variance of components not yet informed by revealed labels. We apply the probit approximation to the logistic–Gaussian integral [7], $p = \sigma\big(m/\sqrt{1 + \pi v/8}\big)$, which keeps early predictions close to the base rate. Questions are matched using their *visible* content, consistent with the runtime interface. This matters because many benchmarks reuse the same prompt for questions that differ only in an attached file that the model cannot access. For example, one benchmark contains 2,960 questions but only 370 distinct texts.

Fitting. We fit all parameters by minimising the training ALC over simulated episodes that reproduce the platform's split and reveal order on training benchmarks. First, we select ℓ and τ on a grid using the own-label offset. We then fit (c, λ) and (τ, σ) using out-of-fold prior logits, followed by ρ and ω on training benchmarks whose pairs are divided into pools of five. The search grids are provided in Appendix B. The fitted parameters remain stable across folds and agree with the observed data. In particular, $\tau = 1.58$ in every fold, compared with the measured logit spread of 1.62. We obtain $\sigma = 0.75$ (0.5 in one fold), $\lambda \in [0.5, 1]$, and $\rho = 0.8$ and $\omega = 1.5$ in every fold. After training on all 160 benchmarks, the submitted parameters are $\lambda = 0.75$, $\tau = 1.58$, $\sigma = 0.75$, $\rho = 0.8$, $\omega = 1.5$.

Runtime. Our submission uses only `numpy` and consists of `model.py`, four runtime modules and fitted parameters stored in a 0.5 MB archive. It requires neither a pretrained model nor network access. Since all targets at a checkpoint receive the same `labeled` list, we compute each benchmark's posterior once per list and cache the result.

4 Results: where the information comes from

Table 1 and Figure 2 show the progression. We take the three sources in the order in which they become available.

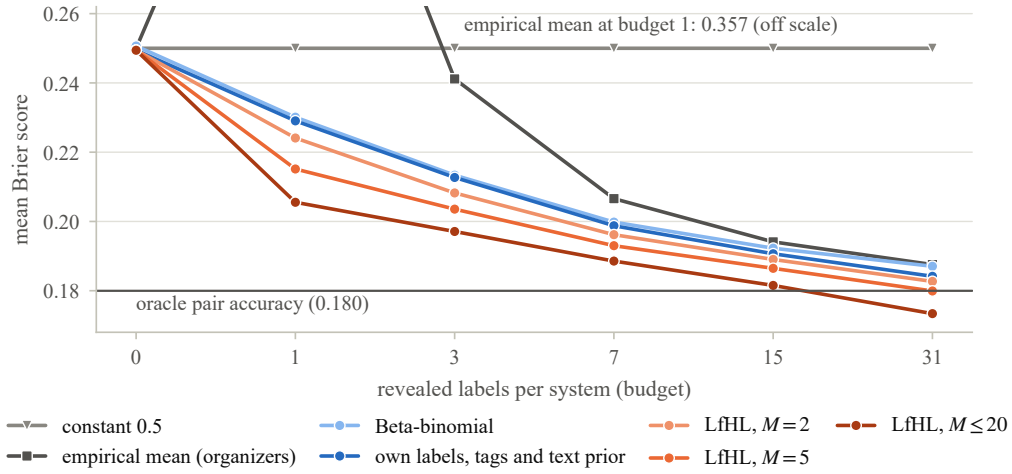


Figure 2: Mean Brier score by budget (same runs as Table 1). The naive empirical mean over-reacts to the first label (0.357 at budget 1, off scale). The Bayesian own-label models move smoothly; pooling labels across systems lowers every curve from budget 1 on, and with up to 20 systems it ends below the pair-accuracy oracle, which no own-label model can do.

4.1 Before any label: little transfers

The logistic text prior performs worse than a constant prediction of 0.5 when used on its own (Figure 3). Furthermore, the regularisation selected on held-out benchmarks is the strongest in our grid for four of five folds. When incorporated into the own-label update, it improves ALC by only 0.0005, with an interval that includes zero. To investigate this result, we train the same features with a separate intercept for each pair, allowing the model to learn differences between questions rather than overall pair accuracy. On new questions from *known* benchmarks, the model achieves an AUC of up to 0.66 (0.59–0.66 across regularisation strengths, 40 benchmarks). However, performance drops to 0.52 on an unseen fold. This suggests that surface features largely identify the benchmark rather than generalisable question difficulty. For example, 79% of the variance in log question length occurs between benchmarks. The meaning of a correct label also differs across benchmarks, with some safety benchmarks treating a refusal as correct. We additionally evaluate gradient-boosted trees [4] on the same features, allowing the model to capture system $\times$ question interactions. However, the fitting procedure assigns them less trust, and they perform worse than the logistic prior in four of five folds.

Two independent studies within our team support the same conclusion (Appendix D). The first evaluates system metadata using family-grouped folds, including release dates imputed for undated systems and identities recovered from model keys. None of the features improves ALC in four of five folds. Exact release dates even worsen performance ($+0.0025$), as they effectively identify individual systems. The second study evaluates a two-way ability $\times$ difficulty model against a model-free baseline that combines other systems' labels on the benchmark with the target pair's own labels. The model performs worse than the baseline (ALC 0.2036 against 0.2004), while its budget-0 predictions are also worse than a constant prediction of 0.5.

We further investigate whether language models can provide useful estimates of question difficulty. On 5,843 text questions from 40 benchmarks, we evaluate Qwen2.5-1.5B and 3B [10] alongside two hosted models, gpt-4.1-mini and gpt-4.1 (Appendix C). Each model rates the difficulty of individual questions. Even when given the benchmark's true level, these ratings explain only about 2% of the remaining gap between a level-only prediction and a question oracle. The hosted models do not outperform the smaller models. However, this evaluation covers only a few models and one prompt, which we discuss further in Section 6.

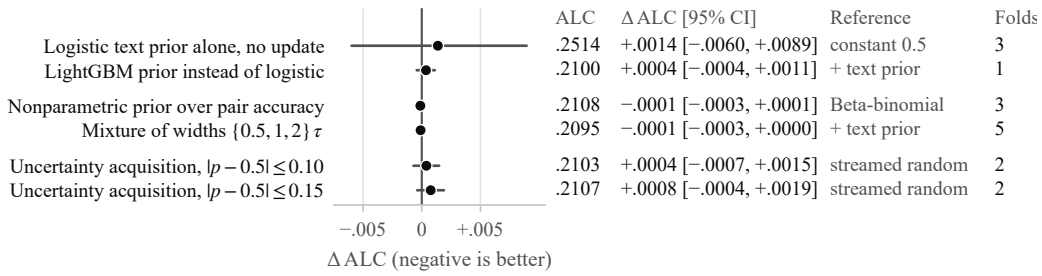


Figure 3: Negative results under the same protocol (1,720 pairs): paired difference in ALC against the reference named in each row, with a 95% bootstrap interval over benchmarks, and the number of folds where the row is better. Every interval includes zero. Streamed random acquisition, the reference of the last two rows, scores .2099 ($+.0003$ [$-.0010$, $+.0016$] against the fixed random order, one fold).

4.2 A system's own labels: a large first gain, then saturation

Reacting to labels must be calibrated. The organizers' empirical-mean baseline predicts accuracy directly from the revealed labels. After one label, this produces a prediction of either 0 or 1, resulting in a Brier score of 0.357 at budget 1, substantially worse than making no update. In contrast, any Bayesian update addresses this problem. The Beta-binomial outperforms the baseline on 75% of pairs and in every fold (-0.0327), achieving a total gain of 0.039 over a constant prediction of 0.5. This represents by far the largest individual improvement in our results.

The update rule is saturated. We compare three approaches to modelling pair accuracy: the Beta prior used by the Beta-binomial, the logit-normal prior of the Bayesian offset and a nonparametric kernel-density prior. All three achieve nearly identical ALC scores of 0.2108–0.2109. Furthermore, combining three prior widths in the full own-label model improves ALC by only 0.0001 (Figure 3). The close agreement across four prior shapes suggests that the remaining error comes from the information available to the update rather than the choice of update rule. Tag shifts provide the only useful additional improvement (-0.0008, five of five folds, from budget 7 on). Unlike global tag weights, these shifts are estimated directly from the revealed questions of the target benchmark.

Choosing which labels to buy does not help. We evaluate an acquisition strategy that prioritises questions with predicted probabilities closest to 0.5. This corresponds to the official baseline's acquisition rule and a classic active-learning heuristic [13]. However, it performs slightly worse than random sampling at both widths, with improvements in only two folds. With a single system, the main objective is to estimate the pair's overall level, for which any representative sample of labels is equally informative. Selecting questions based on their proximity to 0.5 under a weak text prior instead makes the sample less representative. Our simulation also allows skipped candidates to return at the next checkpoint, whereas the platform removes them from the stream. The acquisition policy is therefore more constrained on the actual platform. We submit no `labeling.py`.

4.3 Other systems' labels: a different kind of evidence

Sharing labels across systems evaluated on the same benchmark improves performance. With one other system, the gain is 0.0029; with five, it reaches 0.0070; and with up to 20, it increases to 0.0128, with improvements in all five folds and on 78% of pairs. At budget 31, LfHL with up to 20 systems achieves a score of 0.1734, outperforming the oracle that knows each pair's true accuracy (0.1800). This is impossible for a model that estimates only one number per pair. With one system, LfHL reproduces the own-label results. Repeated question texts slightly affect 148 of 1,720 pairs, changing the mean by less than 10^{-4}. As a result, pooling introduces no cost when `labeled` contains no labels from other systems.

Where the gain comes from. Figure 4 shows the contribution of each shared term. The benchmark level accounts for most of the improvement across all values of M. Using only tags and the benchmark

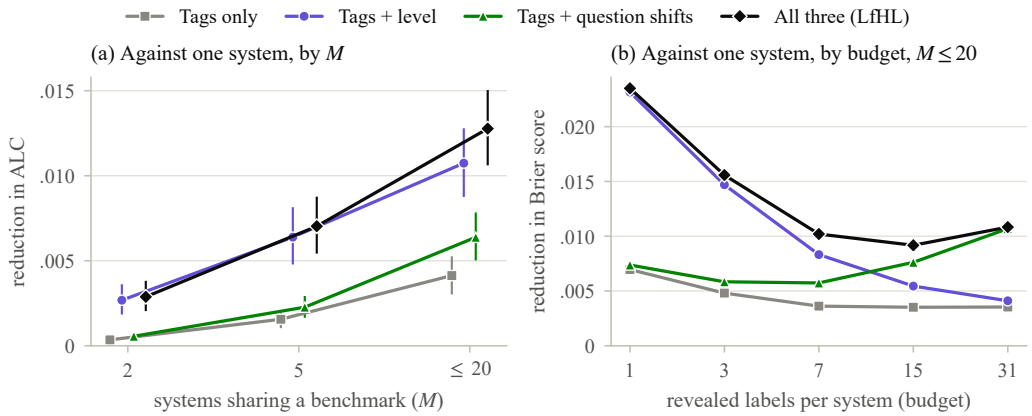


Figure 4: Pooling ablation: the same fitted models with shared terms switched off and nothing refitted. Tags stay on throughout because the one-system model uses them. (a) Reduction in ALC against LfHL with one system (0.2096), with 95% bootstrap intervals over benchmarks. (b) Reduction in Brier score at each budget with up to 20 systems: the benchmark level carries the gain early, question shifts late. Exact values are in Table 2 (Appendix B).

level retains 0.0027 of the 0.0029 gain at $M = 2$, 0.0064 of 0.0070 at $M = 5$, and 0.0107 of 0.0128 at $M \leq 20$. In contrast, question shifts require another system to have revealed the same question. Their contribution is therefore small when few systems are available (-0.0006 at $M = 2$, compared with -0.0003 for tags alone), but increases with M. The results across budgets further illustrate this difference. At budget 1 with $M \leq 20$, where a single own label provides little information, the benchmark level reduces the Brier score from 0.2290 to 0.2059, compared with 0.2055 for the full model. Question shifts reach only 0.2217, compared with 0.2221 for tags alone. However, at budget 31, question shifts become more useful, achieving 0.1735 against 0.1734 for the full model. Tags and the benchmark level reach only 0.1801, close to the oracle that knows each pair's true accuracy (0.1800). These results show that other systems' labels initially provide information about overall benchmark difficulty, while at higher budgets they help identify which individual questions are difficult.

Why other systems can help when features cannot. Question difficulty is shared across systems evaluated on the same benchmark. We find that the mean accuracy of other systems on a question predicts whether a system answers it correctly with an AUC of 0.87 (sample of 40 benchmarks). A separate Rasch analysis (Appendix D) estimates question difficulty using one half of the systems on each benchmark and evaluates it on the remaining half. This achieves an AUC of 0.77, above chance on all 24 benchmarks tested. The analysis also shows that question difficulties vary considerably more than system abilities, with median standard deviations of 3.68 and 0.94 logits, respectively. In contrast, text features achieve an AUC of only 0.52 on unseen benchmarks. Although these three results come from different experimental settings and are not directly comparable, they suggest that the difficulty of a question can be observed from other systems' answers more easily than it can be predicted from its text. However, this information is available only when another system has revealed the exact question. In the platform-sized draws of the Rasch analysis, this occurs for about one in nine evaluation questions even at budget 31 (median over folds).

5 Selected submission

We select LfHL as described in Section 3, fitted on public data only, without a `labeling.py` (Codabench submission #1019997). Development-platform runs score only eight or nine pairs in our logs, so the relevant gain there is that of $M = 2$ (0.0029) rather than $M \leq 20$ (0.0128). Identical uploads of one archive vary with a standard deviation of 0.0185 (47 uploads), so single scores cannot rank models. During development we also explored a variant built from pair accuracies estimated

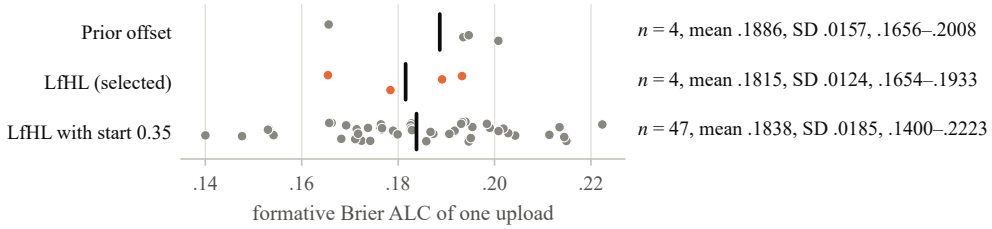


Figure 5: Formative Brier ALC of our own uploads. Each row is one unchanged archive uploaded repeatedly; dots are uploads and bars are means, and the right column gives the number of scored uploads, the mean, the standard deviation and the range. The prior offset uses own labels and the text prior; the last archive has no text prior, and five further uploads of it finished without a score.

from scoring feedback; we asked the organizers whether this is permitted, did not select it, and report no statistics from it.

6 Limitations

- **Overlap between systems.** Our pooled evaluation assumes a fixed number M of systems per benchmark. However, the number of systems in the final test is unknown, and the improvement depends on how many systems share a benchmark (Section 5).
- **Public versus platform pairs.** We fit all parameters on public benchmarks, which also determine the starting level of our selected model. Platform scores (Figure 5) and budget-0 results suggest that the hidden pairs differ from the public data. As a result, improvements observed locally may not transfer directly to the platform.
- **Local protocol.** We group our cross-validation folds by benchmark rather than benchmark family (e.g. the ten `medhelm_*` benchmarks). Since our improvements primarily come from labels revealed at test time rather than learned features, this should have little effect, although family-grouped folds would provide a stricter evaluation. To reduce computational cost, we evaluate at most 20 pairs per benchmark and 200 questions per pair, using a single seed for splits and folds. The platform does not publish its candidate order, so we simulate the acquisition process.
- **Ablation.** We disable individual terms without refitting the remaining parameters, preventing them from compensating for the removed components. Tag shifts are also included in every row.
- **Question-level information.** The question oracle achieves a score of 0.133, substantially below the performance of our models. Pooling captures only part of this information, since it requires other systems to have revealed the same questions. None of the information sources we tested provides more than marginal transfer for a single system. However, we did not evaluate sentence embeddings or open models with tens of billions of parameters, and our predictor does not use a language model. Our language-model study is limited to 40 benchmarks, four models and one rating prompt. Larger models or different prompts may capture considerably more information.
- **Team studies.** The additional studies in Appendix D, apart from the prior variants, use separate evaluation protocols. Furthermore, our within-pair diagnostics of the text prior cover only one fold.

7 Conclusion

Our results show that on unseen benchmarks, the most useful information comes from revealed labels rather than features learned from other benchmarks. A calibrated update of a single shift per pair captures most of the information provided by a system's own labels, with four different prior shapes showing that further improvements are limited. In contrast, labels from other systems provide additional information about the benchmark's overall difficulty and, as more labels become available, the difficulty of individual questions. LfHL uses this information when other systems are available, while reducing to the own-label model when they are not. These findings suggest that, for evaluation design, increasing the overlap between systems on a benchmark may be more useful

than developing a more complex predictor. Beyond the proposed model, we hope that our evaluation protocol, negative results and analysis of score noise will also be useful for future work.

Reproducibility. We release the code, tests and scripts used to generate the tables and figures at `https://github.com/eliashuehne/paiec` (branch `main`). Appendix A provides the commands, run times and seeds. We verify the reproducibility of our submission from a fresh clone, successfully completing installation, all 286 tests and both archive builds. The official checker accepts the resulting archives, and the rebuilt selected archive matches the uploaded version, with identical code apart from line endings and fitted arrays differing by at most 3×10^{-14}. This verification uses an existing copy of the data rather than downloading it again. We additionally re-run the pooled model at $M = 2$ using the ablation script and find that all reported numbers remain unchanged, with the largest difference on a single score being 1.2×10^{-4} and the mean ALC difference remaining below 10^{-6}.

References

[1] AIMS Foundations. measurement-db: Item-level responses of AI systems on public benchmarks. Hugging Face dataset `aims-foundations/measurement-db`, revision `54074453`, 2026.

[2] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78 (1):1–3, 1950.

[3] Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975.

[4] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[5] John P. Lalor, Hao Wu, and Hong Yu. Building an evaluation scale using item response theory. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 648–657, 2016.

[6] Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1980.

[7] David J. C. MacKay. The evidence framework applied to classification networks. *Neural Computation*, 4 (5):720–736, 1992.

[8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[9] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[10] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[11] Georg Rasch. *Probabilistic Models for Some Intelligence and Attainment Tests*. Danish Institute for Educational Research, Copenhagen, 1960.

[12] Pedro Rodriguez, Joe Barrow, Alexander Miserlis Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative: How should that change NLP leaderboards? In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, pages 4486–4503, 2021.

[13] Burr Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison, 2009.

[14] Stanford AIMS Lab. Predictive AI Evaluation Competition (PAIEC). NeurIPS 2026 Competition Track, `https://aims.stanford.edu/competition`, 2026.

[15] Luke Tierney and Joseph B. Kadane. Accurate approximations for posterior moments and marginal densities. *Journal of the American Statistical Association*, 81(393):82–86, 1986.

[16] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 1113–1120, 2009.

A Reproducing the results

All commands run from the repository root with Python 3.10 (`numpy`, `pandas`, `pyarrow`; `scikit-learn`, `scipy` and `lightgbm` for training; `matplotlib` for the figures). The data download needs a Hugging Face account with access to the gated Measurement DB.

```
git clone https://github.com/eliashuehne/paiec && cd paiec
git submodule update --init  # baseline at cc88a95
python -m venv .venv && source .venv/bin/activate    # or .venv\Scripts\activate
pip install -e ".[dev]" matplotlib
hf auth login && python scripts/download_data.py     # revision 54074453, ~5 GB
python -m pytest                                     # 286 tests

# Table 1, Figures 1-2 (five folds, seed 0; 0.5-3 h per command on a laptop CPU)
python scripts/run_eval.py --predictors constant empirical_mean shrunk_mean \
    offset tag_offset
python scripts/run_eval.py --predictors prior_offset gbdt_offset logreg_prior
python scripts/run_eval.py --predictors pooled_offset --pool-sizes 1 2 5 all
# Figure 3, acquisition rows
python scripts/run_eval.py --predictors prior_offset --acquisition-items 100 \
    --policies random uncertainty10 uncertainty15
# Figure 4, Table 2 and the start-0.35 variant (about 35 min, one process per fold)
python scripts/run_pooling_ablation.py --folds 0 1 2 3 4
python scripts/run_pooling_ablation.py --merge

# Selected archive (training on all 160 benchmarks: 11-15 min)
python scripts/train_submission.py      # artifacts/pooled_offset.npz
python scripts/build_submission.py      # dist/submission.zip, official checker

# Numbers and figures of this report (set the run folders in RUNS first)
python report/make_figures.py
```

In the code, LfHL is the predictor `pooled_offset`. Each run of `run_eval.py` writes per-pair scores to `runs/<timestamp>/pairs.csv`. From these, `report/make_figures.py` computes the bootstrap intervals (4,000 resamples of benchmarks, seed 0) and the figures, and writes `report/results/numbers.json`. The data facts of Section 2 and the diagnostics of Section 4.1 come from `scripts/analysis/`, the language-model pilot from `scripts/pilot/`. The nonparametric prior of Figure 3 is on branch `logreg` (commit `aa0f0b2`); the mixture and budget-0 prior variants are on branch `model/subject-features-enhanced` (commit `39f93d2`) as predictors `mix_prior_offset` and `zero_prior_offset`. All experiments ran on one laptop (Intel Core Ultra 9 185H, 32 GB RAM); only the language-model pilot used its GPU (NVIDIA RTX 4060 Laptop).

B Implementation details

Episodes. Fitting replays the platform protocol on training benchmarks: for at most 20 randomly chosen pairs per benchmark (each benchmark contributes at most 20 pairs of weight), the pair's questions are split with the same seeded 50/50 rule, the first 31 acquisition questions are the revealed labels, and at most 200 evaluation questions are scored. Every candidate parameter value is scored by the exact ALC of these episodes, with vectorised implementations that the tests check against the runtime code.

Grids. Own-label offset: 31 levels in $[-1.5, 1.5] \times 41$ values of τ in $[0.1, 10]$ (log-spaced). Beta-binomial: 81 values of α in $[0.05, 200]$. Text prior: $c \in \{-0.2, 0, 0.2\} \times \lambda \in \{0, 0.25, 0.5, 0.75, 1, 1.5\}$ at $\sigma = 0.75$, then $\tau \times \{0.7, 0.85, 1\} \times \sigma \in \{0.5, 0.75, 1\}$. Pooling: $\rho \in \{0, 0.2, 0.35, 0.5, 0.65, 0.8, 0.9, 0.95\} \times \omega \in \{0, 0.5, 1, 1.5, 2, 3\}$ on pools of five. Logistic regularisation $C \in [0.003, 1]$ chosen on an inner 20% of training benchmarks ($C = 0.01$ in the submission).

Tag keys. A key is used if it has at least four revealed questions, two or more values and repeating values; long or hash-like values are dropped.

Features. Item: log characters, words and lines; digit and non-ASCII share; code, mathematics, JSON, question mark, empty content; detected answer options and the implied chance rate; interactors and tags present; hashed unigrams and bigrams of the first 2,000 characters, tag keys and `key=value` pairs, interactor words, in 2^{16} slots. System: release-year offset, reasoning-effort level, parameter count parsed from names, missing-value flags; hashed provider, harness and version, effort, name words and extra settings. Each hashed token has value $1/\sqrt{\text{count}}$.

Learned parameters per fold. $\ell = 0$ (fold 3: -0.1), $c = 0$, $\tau = 1.58$, $\sigma = 0.75$ (fold 3: 0.5), $\lambda = 0.5, 0.5, 1.0, 1.0, 0.5$, $\rho = 0.8$, $\omega = 1.5$; Beta-binomial $p_0 \in [0.49, 0.51]$, $\alpha \approx 2.57$ (method-of-moments estimate 2.4).

Table 2: Exact values of the pooling ablation (Figure 4). Top: change in ALC against LfHL with one system (0.2096), with a 95% bootstrap interval over benchmarks; all cells are better in all five folds, except the two $M = 2$ cells without the level (four folds). Bottom: Brier score by budget with up to 20 systems.

Shared terms	$M = 2$	$M = 5$	$M \leq 20$
Tags only ($\rho = 0, \omega = 0$)	$-.0003$ [$-.0005$, $-.0002$]	$-.0016$ [$-.0021$, $-.0010$]	$-.0041$ [$-.0053$, $-.0030$]
Tags + level ($\omega = 0$)	$-.0027$ [$-.0036$, $-.0018$]	$-.0064$ [$-.0081$, $-.0048$]	$-.0107$ [$-.0128$, $-.0088$]
Tags + question shifts ($\rho = 0$)	$-.0006$ [$-.0008$, $-.0004$]	$-.0023$ [$-.0029$, $-.0016$]	$-.0064$ [$-.0078$, $-.0050$]
All three (LfHL)	$-.0029$ [$-.0038$, $-.0020$]	$-.0070$ [$-.0088$, $-.0054$]	$-.0128$ [$-.0150$, $-.0106$]

Shared terms, $M \leq 20$	BS_1	BS_3	BS_7	BS_{15}	BS_{31}
None (LfHL, $M = 1$)	.2290	.2127	.1988	.1907	.1842
Tags only	.2221	.2079	.1951	.1872	.1807
Tags + level	.2059	.1980	.1904	.1853	.1801
Tags + question shifts	.2217	.2068	.1930	.1831	.1735
All three (LfHL)	.2055	.1971	.1886	.1815	.1734

C Language-model difficulty pilot

We sampled 5,843 questions with plain text (no attached-file stubs) from 40 public benchmarks with at least eight systems each; the target is a question's facility, the share of systems answering it correctly. Qwen2.5-1.5B-Instruct and Qwen2.5-3B-Instruct ran in fp16 (3 forward passes per question, 36 minutes for 1.5B) and produced four signals: a self-rated difficulty from 1 to 9 (expected value over the digit tokens), the entropy and top probability of the first answer token, and the question's negative log-likelihood. Within-benchmark Spearman correlations with facility were -0.12 (1.5B, expected sign in 68% of benchmarks) and -0.14 (3B, 82%) for the rating, and below 0.1 in magnitude for the other signals. For leave-one-benchmark-out evaluation we fitted a linear map from the standardised signals to the centred logit facility on the other benchmarks (slope shrunk by half), and set the held-out benchmark's level so that the mean prediction equals its true mean facility, which isolates the ordering of questions. The expected Brier score of a random system fell from 0.1945 (level only) to 0.1930 (1.5B) and 0.1932 (3B), against 0.1302 for the per-question oracle: 2% of the available room. Using the mean rating to predict a benchmark's level was inconsistent (-0.003 with 1.5B, $+0.003$ with 3B, against a constant level). We repeated the rating with gpt-4.1-mini and gpt-4.1 through the OpenAI API (same rating prompt, first 3,000 characters of the question, temperature 0, expected digit from the top 20 next-token probabilities). They gave a digit for 5,715 and 5,667 of the questions; on those, the within-benchmark Spearman correlation is -0.18 for both (expected sign in 80% and 72% of benchmarks) and the expected Brier score falls from 0.1941 to 0.1927 and 0.1930, against 0.1301 and 0.1297 for the question oracle (2% of the room). Scripts are in `scripts/pilot/`. All these models were used offline only and are not part of the submission.

D Team studies of system and question information

The first three studies were run by team members with their own harnesses on the public Measurement DB; their protocols differ from Section 2, so their numbers are not directly comparable with Table 1 and Figure 3. The prior variants at the end were scored with our protocol. Code and full reports are in the repository: E.H.'s on branch `feature-foundation`, Q.G.'s metadata screen on `main`.

Is question difficulty shared? (E.H., `docs/competition-item-axis.md`) A penalised Rasch model, $\operatorname{logit} p = \theta_s - b_q$, was fitted per benchmark on 37 benchmarks with question-level data (517,191 questions). Splitting each benchmark's systems into two disjoint halves, the facility of a question correlates at 0.670 between the halves (positive in all 24 benchmarks with an estimate), and difficulty learned from one half ranks the other half's individual answers with AUC 0.765 (0.587 to 0.965, above chance in all 24 benchmarks). The median standard deviation of question difficulty is 3.68 logits, against 0.94 for system ability. In oracle terms on that harness, knowing each evaluation question's recorded accuracy scores ALC 0.0657, against 0.1764 for knowing each pair's accuracy. Revealed labels reach little of it: the share of evaluation questions that some other system in the draw has revealed is 0.07, 0.50, 0.11, 0.01 and 0.76 at budget 31 in the five folds, so an oracle for the facility of revealed questions (0.1723) barely beats the pair-accuracy oracle (0.1764). Question difficulty is therefore a property of the question, and the large remaining prize is to predict it from content, which our text features (Section 4.1) and the language-model pilot (Appendix C) did not achieve.

An ability $\times$ difficulty model (E.H., `docs/competition-pairs.md`) On 6,252 pairs of 41 text-only benchmarks with a platform-style scorer (50/50 split, nested budgets, labels shared across pairs), a hierarchical model with a system ability, a benchmark difficulty, a text-based difficulty regression and a Bayesian update scored ALC 0.2036, against 0.2004 for a model-free baseline that combines the benchmark's other labels with the pair's own, and beat that baseline in one fold of five. Its budget-0 Brier score was 0.2544 (0.2902 with the text difficulty term), worse than predicting 0.5. Ability alone scored 0.2012.

System metadata (Q.G., `docs/subjects.md`) A screening harness adds one feature at a time to a gradient-boosted correction of a revealed-label baseline (ALC 0.2034), with benchmark folds grouped by benchmark family and a sealed holdout, at all six budgets, and reports the change in ALC with a bootstrap interval over 125 held-out benchmarks (a column of random noise costs $+0.0003$). Release dates were parsed and, for undated systems, estimated from version strings with references from other folds only; system identity was recovered from model keys in the setting fields, raising coverage from 65.1% to 89.4% of rows. No column improved ALC by 0.001 in four of five folds. Exact release dates hurt ($+0.0025$ [$+0.0013$, $+0.0038$], no fold better), because they are nearly unique per system; coarse or monotone forms were neutral ($+0.0002$ [-0.0003, $+0.0007$]), as were imputed dates (-0.0002 [-0.0004, $+0.0001$] against real dates only), parameter count, provider and model family, and ability matched by recovered identity ($+0.0003$ [-0.0001, $+0.0007$]). The effect of recency is real but small: within a benchmark a system one year newer scores 7.1 points higher, yet release years within a benchmark have a standard deviation of only 0.45 years, which bounds the gain at 0.0005–0.0006 ALC; and before the first label, benchmarks that test newer systems are harder (correlation -0.11), so recency misleads at budget 0.

Prior variants (Q.G., branch `model/subject-features-enhanced`, commit `39f93d2`) Two variants of the own-label model with the text prior, scored with our protocol. The *mixture* (Figure 3) replaces the single pair-shift prior by $\sum_m w_m \mathcal{N}(0, (m\tau)^2)$, $m \in \{0.5, 1, 2\}$, with posterior weights from each component's Laplace evidence, so that a pair whose labels are far from the prior can move faster. The *budget-0 shift and scale* gives the prior logit its own shift and scale when no label is revealed (shift in $\{-0.4, \ldots, 0.4\}$, scale in $\{0, \ldots, 1\}$; scale 0 is a constant): ALC .2096, $-.0000$ [$-.0001$, $+.0000$] against the text prior, better in two folds.

E Disclosures

Training data and models. The submission is trained only on the 160 public benchmarks of Measurement DB (revision `54074453`). It uses no pretrained model. Qwen2.5-1.5B/3B-Instruct, gpt-4.1-mini and gpt-4.1 were used offline in the pilot of Appendix C only.

Formative feedback. We used formative scores of our own uploads to measure score noise and to check runtime behaviour. Section 5 discloses a further analysis of scoring feedback and the variant built from it, which we did not select.

Use of AI assistants. We used Claude Code (Anthropic) and Codex (OpenAI) for code implementation, unit testing and refactoring. The research methodology and experimental design were developed by the authors through joint discussions, with Claude also used to support brainstorming. We further used Claude for assistance with parts of the report's writing. All AI-assisted code and text were reviewed and revised by the authors, who take full responsibility for the methodology, results and final manuscript.

Contributions. A.D. designed and implemented the predictors, the fitting procedures, the pooled and acquisition evaluation, the submission and the analysis of formative scores, and wrote the report. E.H. set up the repository and the original evaluation harness (metrics, split, pairs, evaluation) on which ours builds, explored neural and gradient-boosted cold-start predictors, and ran the studies of shared question difficulty and of the ability $\times$ difficulty model (Appendix D). Q.G. built the feature-screening harness with grouped folds and sealed benchmarks, the system-identity and release-date features and their screen, and the mixture and budget-0 prior variants (Appendix D). All authors discussed the results and edited the report.